Docker Notes, Cheatsheet & Best Practices

- Docker — Notes, Cheatsheet & Best Practices for DevOps
  - 1. What Docker Is & Why It Matters
  - 2. Docker Architecture (brief)
  - 3. Installation & Setup
  - 4. Images — Cheatsheet
  - 5. Containers — Cheatsheet
  - 6. Dockerfile Essentials
  - 7. Volumes & Networking — Cheatsheet
  - 8. Docker Compose — Cheatsheet
  - 9. Cleaning Up — Cheatsheet
  - 10. Debugging Tips
  - 11. Docker Best Practices for DevOps Engineers
  - 12. Quick Reference Table

Docker — Notes, Cheatsheet & Best Practices for DevOps

---

1. What Docker Is & Why It Matters

Docker packages an application with everything it needs to run — code, runtime, libraries, system tools — into a single **image**. That image runs as a **container**: an isolated, lightweight process that behaves the same on your laptop, in CI, and in production. This solves the classic "it works on my machine" problem and is foundational to modern DevOps — CI/CD pipelines, Kubernetes, and most cloud deployments are built around containers.

Key concepts: | Term | Meaning | |—|—| | **Image** | A read-only template/blueprint (built from a Dockerfile) | | **Container** | A running (or stopped) instance of an image | | **Dockerfile** | A text file with instructions to build an image | | **Registry** | Where images are stored/shared (Docker Hub, ECR, GCR, etc.) | | **Volume** | Persistent storage that outlives a container | | **Network** | Virtual network connecting containers | | **Docker Compose** | Tool for defining/running multi-container apps via YAML |

---

2. Docker Architecture (brief)

- **Docker Client** — the `docker` CLI you type commands into
- **Docker Daemon (`dockerd`)** — background service that builds/runs/manages containers
- **Docker Registry** — stores images (Docker Hub is the default public one)

Containers share the host's kernel (unlike VMs, which virtualize an entire OS) — this is why containers are much lighter and faster to start than virtual machines.

---

3. Installation & Setup

```
docker --version                # check installed version
docker info                     # detailed system/daemon info
docker version                  # client & server version info
sudo systemctl start docker     # start the Docker daemon (Linux)
sudo systemctl enable docker    # start Docker on boot
docker run hello-world          # verify installation works
```

Best practice: on Linux, add your user to the `docker` group (`sudo usermod -aG docker $USER`) so you don't need `sudo` for every command — but understand this grants root-equivalent access, so only do it on trusted machines.

---

4. Images — Cheatsheet

```
docker images                          # list local images
docker pull nginx:1.25                 # download an image (always pin a tag!)
docker pull nginx:latest               # avoid 'latest' in production — see best practices
docker build -t myapp:1.0 .            # build an image from a Dockerfile in current dir
docker build -t myapp:1.0 -f Dockerfile.prod .   # build from a specific Dockerfile
docker tag myapp:1.0 myrepo/myapp:1.0  # tag an image for a registry
docker push myrepo/myapp:1.0           # push image to a registry
docker rmi myapp:1.0                   # remove an image
docker image prune                     # remove unused (dangling) images
docker image prune -a                  # remove ALL unused images (not just dangling)
docker history myapp:1.0               # show image build layers
docker inspect myapp:1.0               # detailed metadata (JSON)
docker save -o myapp.tar myapp:1.0     # export image to a tar file
docker load -i myapp.tar               # import image from a tar file
```

---

5. Containers — Cheatsheet

```
docker run nginx                           # run a container (foreground)
docker run -d nginx                        # run detached (in background)
docker run -d --name web nginx             # run detached with a custom name
docker run -d -p 8080:80 nginx             # map host port 8080 -> container port 80
docker run -it ubuntu bash                 # interactive terminal session
docker run -e "ENV=production" myapp       # set an environment variable
docker run -v /host/path:/container/path myapp  # mount a volume/bind mount
docker run --rm myapp                      # auto-remove container when it stops
docker run --restart unless-stopped myapp  # auto-restart policy

docker ps                                  # list running containers
docker ps -a                               # list ALL containers (incl. stopped)
docker start <container>                   # start a stopped container
docker stop <container>                    # gracefully stop (SIGTERM, then SIGKILL after timeout)
docker restart <container>                 # restart a container
docker kill <container>                    # force-stop immediately (SIGKILL)
docker rm <container>                      # remove a stopped container
docker rm -f <container>                   # force-remove (even if running)
docker container prune                     # remove all stopped containers

docker logs <container>                    # view container logs
docker logs -f <container>                 # follow/tail logs live
docker logs --tail 100 <container>         # last 100 lines only
docker exec -it <container> bash           # open a shell inside a running container
docker exec <container> ls /app            # run a one-off command inside container
docker inspect <container>                 # detailed metadata (JSON)
docker stats                               # live CPU/memory/network usage of containers
docker top <container>                     # processes running inside a container
docker cp file.txt <container>:/app/file.txt # copy a file into a container
docker cp <container>:/app/log.txt ./log.txt # copy a file out of a container
```

---

6. Dockerfile Essentials

```
# Example: Node.js app, multi-stage build
```

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/index.js"]
```

Key instructions: | Instruction | Purpose | |—|—| | `FROM` | Base image to build on | | `WORKDIR` | Set working directory inside the image | | `COPY` / `ADD` | Copy files into the image (`COPY` preferred — `ADD` has extra, often-unwanted behavior like auto-extracting tars) | | `RUN` | Execute a command at build time (installs, etc.) | | `ENV` | Set an environment variable | | `EXPOSE` | Document which port the container listens on | | `USER` | Run as a non-root user | | `CMD` | Default command run when container starts (overridable) | | `ENTRYPOINT` | Fixed command that always runs (args from `CMD` get appended) | | `HEALTHCHECK` | Define how Docker checks if the container is healthy | | `ARG` | Build-time-only variable |

---

7. Volumes & Networking — Cheatsheet

```
docker volume create mydata              # create a named volume
docker volume ls                         # list volumes
docker volume inspect mydata             # details about a volume
docker volume rm mydata                  # remove a volume
docker volume prune                      # remove unused volumes

docker run -v mydata:/var/lib/data myapp   # named volume (managed by Docker)
docker run -v $(pwd):/app myapp            # bind mount (maps a host folder directly)

docker network ls                        # list networks
docker network create mynet              # create a custom network
docker network inspect mynet             # details about a network
docker run --network mynet myapp         # attach a container to a network
docker network connect mynet <container> # connect a running container to a network
```

Note: named volumes are managed by Docker and are the recommended way to persist data (e.g. databases). Bind mounts are great for local development (live code reload) but tie you to the host's filesystem layout.

---

8. Docker Compose — Cheatsheet

```
# docker-compose.yml example
version: "3.9"
services:
  web:
    build: .
    ports:
      - "8080:80"
    environment:
      - ENV=production
    depends_on:
      - db
    restart: unless-stopped
```

```
  db:
    image: postgres:16
    volumes:
      - dbdata:/var/lib/postgresql/data
    environment:
      - POSTGRES_PASSWORD=secret
volumes:
  dbdata:
```

```
docker compose up               # start all services (foreground)
docker compose up -d            # start all services detached
docker compose down             # stop & remove containers/networks
docker compose down -v          # also remove volumes
docker compose build            # build/rebuild images
docker compose ps               # list services & status
docker compose logs -f          # follow logs from all services
docker compose exec web bash    # shell into the 'web' service
docker compose restart web      # restart one service
docker compose config           # validate & print resolved config
```

---

9. Cleaning Up — Cheatsheet

```
docker system df                # disk usage summary (images, containers, volumes, cache)
docker system prune             # remove unused containers, networks, dangling images
docker system prune -a          # also remove all unused images (aggressive)
docker system prune -a --volumes # also remove unused volumes (be careful — data loss)
```

Best practice: run `docker system df` before pruning so you know what you're about to remove, and never `--volumes` prune on a machine with databases you care about.

---

10. Debugging Tips

```
docker logs -f --tail 200 <container>       # tail recent logs live
docker exec -it <container> sh              # shell in (use 'sh' for minimal/alpine images)
docker inspect <container> | grep -A5 "State" # check exit code, health status
docker events                               # real-time stream of Docker daemon events
docker diff <container>                     # see filesystem changes vs the image
```

If a container exits immediately, check `docker logs <container>` first — most crash-on-start issues show up there. If `docker exec` fails because the container isn't running, use `docker run -it <image> sh` to poke around the image itself.

---

11. Docker Best Practices for DevOps Engineers

Image building

1. **Pin base image versions** (`node:20-alpine`, not `node:latest`) — reproducibility matters; `latest` can silently change under you.
2. **Use minimal base images** (`alpine`, `distroless`, `slim` variants) to reduce attack surface and image size.
3. **Use multi-stage builds** to keep build tools (compilers, dev dependencies) out of your final production image.
4. **Order Dockerfile instructions from least to most frequently changing** — put dependency installation before copying source code, so Docker's layer cache is reused on rebuilds and builds stay fast.
5. **Use a `.dockerignore` file** (like `.gitignore`) to keep `node_modules`, `.git`, secrets, and build artifacts out of the build context.

6. **Combine `RUN` commands** with `&&` where sensible to reduce the number of layers.
7. **Don't run as root inside the container** — set a `USER` in the Dockerfile.

Security

8. **Never bake secrets into images** (API keys, passwords) — they persist in image layers even if "deleted" later. Use environment variables, secrets managers, or Docker/Kubernetes secrets at runtime instead.
9. **Scan images for vulnerabilities** (`docker scout`, Trivy, Snyk) as part of your CI pipeline.
10. **Keep base images updated** — rebuild regularly to pick up security patches, don't just "set and forget."
11. **Use read-only filesystems where possible** (`docker run --read-only`) to limit what a compromised container can modify.

Runtime & operations

12. **Always set resource limits** in production (`--memory`, `--cpus`) so one misbehaving container can't starve the host.
13. **Define a `HEALTHCHECK`** so orchestrators (Compose, Kubernetes, Swarm) know when a container is actually ready/healthy, not just "running."
14. **Handle `SIGTERM` gracefully in your app** — Docker sends `SIGTERM` on `docker stop`, and after a grace period (default 10s) sends `SIGKILL`. Apps that ignore this get killed mid-request during deploys.
15. **Log to stdout/stderr**, not to files inside the container — let Docker's logging driver (or your log aggregator) handle collection. Containers are ephemeral; files inside them can vanish.
16. **Use named volumes for anything you need to persist** (databases, uploads) — container filesystems are disposable by design.
17. **Tag images meaningfully** (git SHA, semantic version) rather than relying on `latest` for deployments, so you always know exactly what's running and can roll back precisely.

Workflow

18. **Treat Dockerfiles and Compose files as code** — version control them, review changes via PR, same as application code.
19. **Keep containers single-purpose** — one primary process per container (don't cram a web server, a database, and a cron daemon into one image).
20. **Automate image builds and pushes in CI/CD** rather than building/pushing manually from a laptop, so what's deployed is always traceable to a commit.
21. **Clean up regularly** (`docker system prune`) on build/CI machines — leftover images and volumes silently eat disk space over time.

---

12. Quick Reference Table

Task	Command
List running containers	`docker ps`
List all containers	`docker ps -a`
Run a container	`docker run -d -p 8080:80 --name web nginx`
Stop a container	`docker stop <container>`
Remove a container	`docker rm <container>`
View logs	`docker logs -f <container>`
Shell into container	`docker exec -it <container> bash`
Build an image	`docker build -t myapp:1.0 .`
List images	`docker images`

Remove an image	`docker rmi <image>`
Push image to registry	`docker push myrepo/myapp:1.0`
Start Compose stack	`docker compose up -d`
Stop Compose stack	`docker compose down`
Clean up unused resources	`docker system prune -a`
Check disk usage	`docker system df`
Inspect an object	`docker inspect <name>`